



Comandos SQL

Prof. Sidney Nicolau Venturi Filho

Apresentação

Este conteúdo é fundamental para o profissional de TI, particularmente para os desenvolvedores de sistemas. Todo sistema comercial atual faz amplo uso de banco de dados relacionais. Portanto, conhecer a linguagem SQL nos permite criar, manter e consultar os dados e, assim, contribuir para o funcionamento eficiente do negócio.

Propósito

LOREM_IPSUM

Preparação

Sugerimos que realize o [download](#) dos arquivos utilizados neste conteúdo.

Objetivos

Módulo 1

Projeto físico de banco de dados

Criar e configurar um banco de dados no PostgreSQL.

Módulo 2

Comandos LMD com uma tabela

Consultar dados em uma tabela.

Módulo 3

Comandos LMD com duas ou mais tabelas

Consultar dados em mais de uma tabela.

Módulo 4

Visões e linguagem procedural de banco de dados

Implementar código na utilização de linguagem procedural de banco de dados.



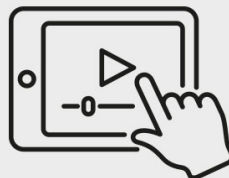
Introdução

O PostgreSQL é um sistema gerenciador de banco de dados relacional dos mais importantes comercialmente, desenvolvido como projeto de código aberto. Sua ampla aceitação no mercado

o torna muito relevante para o profissional de tecnologia da informação que atua na área de banco de dados.

Veremos, de forma bem prática, como realizar o projeto físico de um banco de dados, criar um banco de dados, implementar consultas SQL em uma ou mais tabelas e criar procedimentos armazenados, funções e gatilhos.

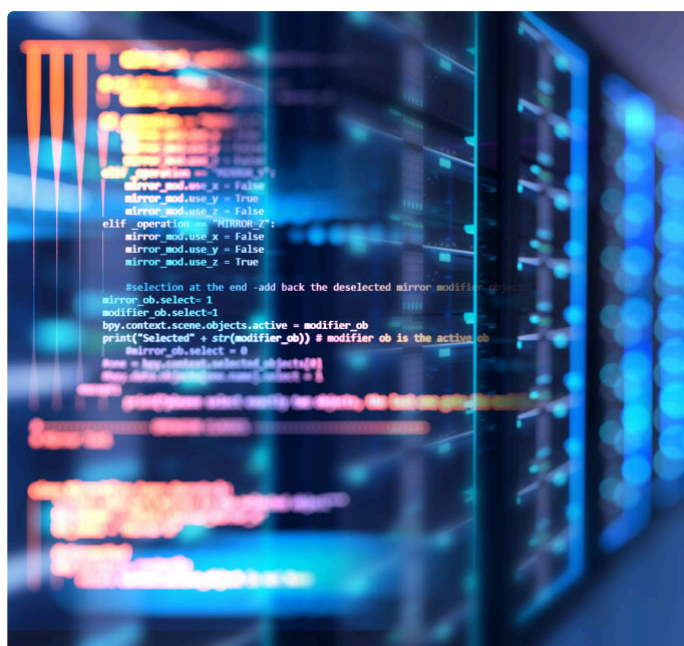
Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

[Download material](#)



1 – Projeto físico de banco de dados

Ao final deste módulo, você será capaz de criar e configurar um banco de dados no PostgreSQL.

Projeto de banco de dados e SQL

O projeto de banco de dados possui três fases, vejamos!

1

Projeto conceitual

Fase de formalização das características relevantes do minimundo utilizando o Diagrama Entidade Relacionamento.

2

Projeto lógico

Fase em que a realidade formalizada é portada para uma classe de tecnologia, no nosso caso o Modelo Relacional.

3

Projeto físico

Fase em que o Modelo Lógico é preparado para ser implementado utilizando um Sistema Gerenciador de Banco de Dados (SGBD) específico a partir da criação de um Script na linguagem SQL.

Neste vídeo, vamos trazer uma visão geral do projeto de banco de dados e da SQL. Você conhecerá alguns conceitos básicos da linguagem SQL considerando um modelo de dados. Confira!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Linguagem SQL

Divide-se em partes, cada uma atendendo a uma necessidade específica. Temos então a seguinte divisão:



Linguagem de Definição de Dados

(Data Definition Language - DDL)

Tendo como principais comandos Create, Alter e Drop.



Linguagem de Manipulação de Dados

(Data Manipulation Language - DML)

Tendo como principais comandos Insert, Update, Delete e Select.



Linguagem de Controle de Dados

(Data Control Language - DCL)

Tendo como principais comandos Grant e Revoke.

Alguns conceitos básicos

Vamos conferir alguns conceitos importantes.

Tabelas



São os objetos básicos de armazenamento de dados no modelo relacional. Elas organizam informações de forma tabular, com colunas representando atributos e linhas contendo registros de dados.

Nome



É normalmente definido durante a modelagem lógica, constituindo, às vezes, alguma variação em relação ao nome da entidade, por exemplo, entidade Aluno vira tabela aluno ou alunos. A maior parte dos SGBD's não dão suporte a caracteres em português no nome da tabela.

Colunas



Originaram-se dos atributos das entidades. Da mesma forma que aconteceu com o nome da tabela, temos que respeitar as limitações do SGBD.

Tipos de dados



As colunas possuem um tipo de dados que pode armazenar, de forma similar ao conceito de tipo utilizado nas variáveis criadas em programas.

O PostgreSQL possui os seguintes tipos básicos que iremos utilizar. Vamos conferir!

`VARCHAR(tam)`

Caracter de tamanho variável, podendo atingir de 1 até 8000 bytes, sendo limitado a tam.

`CHAR(tam)`

Caracter de tamanho fixo, podendo tam ser especificado no máximo de 8000 caracteres.

INTEGER

Número inteiro de 4 bytes podendo ir de -2147483648 a +2147483647.

NUMERIC(I,d)

Numérico com precisão, em i estabelecemos quantos dígitos tem o numero e em d quantos na parte decimal, por exemplo para especificar um número com formato 99999.99. tipo seria NUMERIC(7,2)

DATE

Data entre 01 de janeiro de 4713AC até 31 de dezembro de 32767DC.

Restrições

Normalmente são chamadas *constraint*. Elas visam estabelecer regras para orientar o SGBD na forma de manter a integridade do banco. Vejamos as principais!

NOT NULL (Obrigatório)

Especifica se a coluna é obrigatória para todas as linhas da tabela, não podendo conter nenhum valor NULO.

UNIQUE (Chave Alternada)

A coluna ou combinação de diversas colunas, tem que ser única para todas as linhas da tabela, não permitindo repetições.

PRIMARY KEY (Chave Primária)

É a chave primária de identificação unívoca da tabela. Pode ser uma ou uma combinação de colunas.

FOREIGN KEY (Chave Estrangeira)

É uma coluna que garante a integridade de uma relação entre duas tabelas, sendo referenciada por uma chave primária da outra tabela.

Modelo de dados da empresa

Em nosso estudo, iremos utilizar o banco de dados da empresa cujo modelo lógico se encontra a seguir.

Modelo de dados do estudo de caso.

Atividade 1

Quando vamos criar uma tabela podemos impor restrições às suas colunas. Assinale a alternativa que apresenta a restrição que determina que uma coluna é única de preenchimento obrigatório.

A Foreign Key

B Not Null

C Unique

D Check

E Primary Key

Parabéns! A alternativa E está correta.

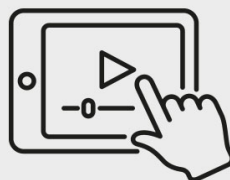
A chave primária serve de identificador da linha da tabela, sendo um campo de preenchimento obrigatório e único.

SGBD PostgreSQL

O PostgreSQL é um sistema de gerenciamento de banco de dados objeto-relacional (SGBDOR) de código aberto. Vamos então ver como realizamos a sua instalação no Windows.

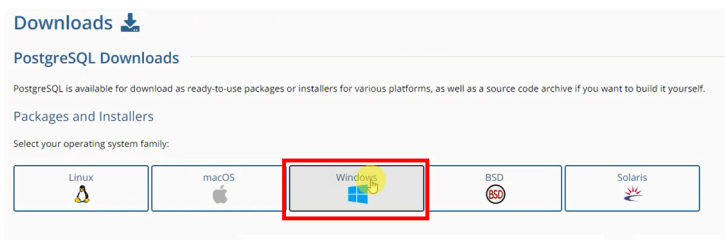
Assista a este vídeo e aprenda a fazer todo o processo de instalação do SGBD PostgreSQL, desde o download do programa ao uso do pgAdmin4.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.

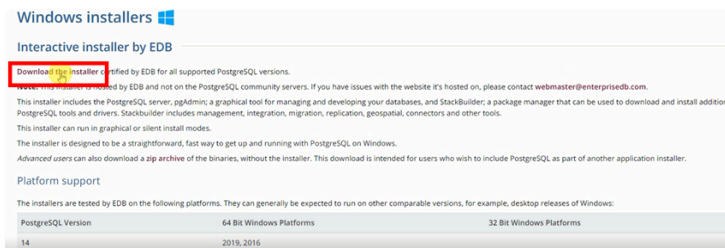


Download do instalador para Windows

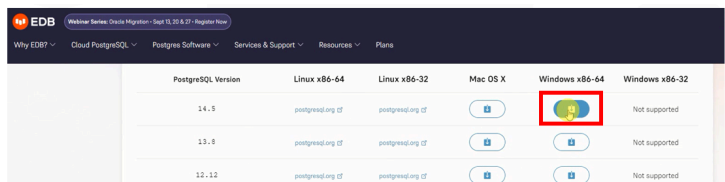
O site para download do PostgreSQL é o www.postgresql.org, nele existem verões do SGBD para uma série de SO. Vamos fazer o download para Windows!



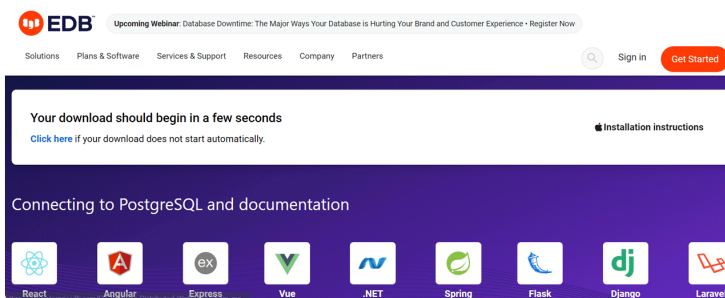
Para iniciar, escolha o sistema operacional Windows.



Clique no link para realizar o download do instalador.



Escolha a versão do SGBD a ser baixado.



Ocorrerá então o download do instalador.

Instalando no Windows

Dê um duplo clique para começar a instalação.

Instalação do PostgreSQL.

Clique no botão Next.

Instalação do PostgreSQL.

A próxima tela permite que você escolha os componentes que vai instalar:



PostgreSQL Server

Servidor propriamente dito.



pgAdmin4

Interface gráfica para administração do banco de dados.



Stack Builder

Software que permite instalar extensões para o servidor.



Command Line Tool

Interface de linha de comando para administração do servidor.

No nosso caso, vamos instalar todos, clique no botão Next.

Instalação do PostgreSQL.

Você pode mudar o diretório ou aceitar o proposto e a seguir deve clicar no botão Next.

Instalação do PostgreSQL.

A próxima tela irá solicitar que você defina a senha do usuário postgres. Esse é o superusuário do SGBD, o DBA principal que tem acesso a todos os dados e recursos do sistema.

Ao definir a senha, não a esqueça. Clique em Next.

Instalação do PostgreSQL.

A porta normalmente utilizada é a 5432. Se você desejar, pode modificá-la colocando qualquer outro número entre 1024 e 65535, mas sugerimos que utilize a padrão, na sequência, clique no botão Next.

Instalação do PostgreSQL.

Na opção default, serão utilizadas as configurações do SO, mas você pode modificar isso escolhendo outra localização na caixa combo. Em qualquer dos dois casos clique no botão Next após a escolha.

Instalação do PostgreSQL.

Clique no botão Next.

Instalação do PostgreSQL.

Ele agora diz que está pronto. Clique no botão Next.

Instalação do PostgreSQL.

Agora é iniciada a instalação.

Instalação do PostgreSQL.

Como no momento não vamos instalar nenhum complemento, desmarque a caixa de seleção e clique no botão Finish.

Instalação do PostgreSQL.

Após a instalação, estará disponível na barra de programas o Stake Build, o pgAdmin4 e os demais componentes do programa.

Barra de programas após instalação do PostgreSQL.

Uso do pgAdmin 4

O pgAdmin 4 é uma ferramenta gráfica de administração do PostgreSQL. Ao instalarmos o SGBD no Windows, a ferramenta é instalada automaticamente. Para abrir, clique em seu ícone na lista de programas.

Acesso ao pgAdmin 4.

A seguir, aparecerá a tela de carga.

Iniciação do pgAdmin4.

Após a carga, aparecerá a tela de login. Preencha a senha que você definiu durante a instalação e clique em OK.

Acesso ao pgAdmin 4.

A seguir clique no servidor, preencha a senha dos postgres e pressione OK.

Conexão ao servidor no pgAdmin 4.

Após isso você estará conectado ao servidor e já poderá acessar e administrar o SGBD.

Conexão ao servidor no pgAdmin 4.

Atividade 2

O PostgreSQL é um software livre e Open Source que pode ser classificado como

- A Sistema de banco de dados.
- B Banco de dados.
- C Sistema Gerenciador de Banco de Dados (SGBD).
- D Base de dados.
- E Interface de acesso a dados.

Parabéns! A alternativa C está correta.

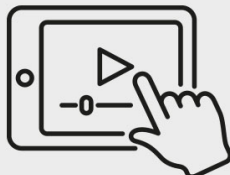
O PostgreSQL foi desenvolvido como um programa especialista em gerenciar bancos de dados, sendo classificado como Sistema Gerenciador de Banco de Dados (SGBD).

Criação de um banco de dados – DDL e DML

A criação de um banco de dados passa pela criação do banco, de suas tabelas e pela inserção das linhas com os dados.

Acompanhe neste vídeo a criação de um banco de dados no PostgreSQL que envolve a execução de comandos no pgAdmin4, a criação e alteração de tabelas, inserção, atualização e eliminação de linhas em tabelas, bem como a eliminação de tabelas e o controle de transações.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Criando um banco de dados no PostgreSQL

Para criar um banco de dados PostgreSQL, faça conexão com o pgAdmin4. Clique com o botão direito em database e selecione Create Database.

Criação de um banco de dados.

Na janela aberta, na aba General, defina o nome do banco.

Criação de um banco de dados.

A seguir, na aba Definition, escolha o valor para os diversos parâmetros de criação.

Criação de um banco de dados.

A aba SQL mostra o comando que será executado, basta então clicar em Save.

Criação de um banco de dados.

Uma vez criada a base, ela aparece na árvore de administração.

Criação de um banco de dados.

Executando comandos pelo pgAdmin4

Abra uma janela de consulta, para isso clique com o botão direito no nome do banco e selecione Query Tool.

Criação de um banco de dados.

Será aberta então a janela para a escrita e execução de consultas SQL.

Criação de um banco de dados.

Criando tabelas

O comando de Criação de Tabela é o Create Table. Vejamos sua sintaxe:

POSTGRESQL



Em que as palavras-chaves para criação de tabela são:



nome_da_tabela

Nome que será atribuído à tabela.

Nome_coln



Nome atribuído à coluna n.



Tipo_coln

Tipo de dado atribuído à coluna n.



restri_coln

Define uma restrição de integridade automática a qual os campos da coluna devem obedecer.



restri_tabn

Define uma restrição de integridade automática a qual toda a tabela deve obedecer.

Vamos ver um exemplo de criação. Considere a tabela EMPREGADO do banco de dados da empresa.

Exemplo: tabela EMPREGADO.

Vejamos o comando de criação!

Criação de tabela.

Para podermos criar as chaves estrangeiras, a tabela referenciada deve ter sido criada antes. No comando acima, já teríamos que ter criado a tabela DEPARTAMENTO antes de criar EMPREGADO.

Criação de tabela.

Vamos então criar a tabela DEPARTAMENTO!

Criação de tabela.

Agora, se comandarmos a criação de EMPREGADO, dará certo.

Criação de tabela.

As tabelas irão aparecer na lista de tabelas, confira!

Criação de tabela.

Alterando tabelas

Se olharmos o modelo da empresa podemos notar que a tabela DEPARTAMENTO possui três colunas e uma delas é chave estrangeira para REGIAO.

Modelo de dados do estudo de caso.

A tabela que criamos possui duas colunas apenas, vamos agora acrescentar uma coluna em departamento chamada ID_REGIAO Numeric(7). Para isso, vamos utilizar o comando ALTER TABLE, cuja sintaxe é:

POSTGRESQL



Em que:

- <nome_tabela> é o nome da tabela.
- <nome coluna> é o nome da coluna.
- <tipo da coluna> é o tipo de dado da coluna.
- <constraint> é a restrição, se for o caso.

O comando fica então:

Adicionando uma coluna na tabela.

Para confirmar, dê um Select na tabela.

Adicionando uma coluna na tabela.

Observação: O comando Select acima mostra o conteúdo da tabela, no caso, sua estrutura e nenhuma linha. E agora podemos criar a chave estrangeira.

Primeiro vamos criar a tabela REGIAO.

Criação de tabela.

Para criarmos a chave estrangeira em DEPARTAMENTO, utilizaremos novamente ALTER TABLE com a seguinte sintaxe:

POSTGRESQL



Em que:

- <TABELA > – é a tabela a ser alterada.
- ADD FOREIGN KEY – é a restrição a ser acrescentada.
- (COLUNA) – é a coluna que receberá a constraint.
- REFERENCES TABREF(COLUNA) – indica a tabela e coluna referenciadas pela chave estrangeira.

Confira como fica o comando!

Adicionando chave estrangeira à tabela.

Observação: Para evitarmos a criação de tabelas em determinada ordem, devido às chaves estrangeiras, é muito comum em scripts de criação de banco de dados criarmos as tabelas somente com as colunas, chaves primárias, not null e unique e, após a criação de todas as tabelas, utilizarmos ALTER TABLE para criar as chaves estrangeiras.

Inserindo linhas nas tabelas

O comando INSERT insere linhas em uma tabela. Sua sintaxe básica é:



Em que as cláusulas do comando INSERT são:

- nome_tabela – é o nome da tabela a ser atualizada.
- Coluna n – é a coluna que queremos inserir.
- Valor n – é o novo valor associado à coluna a ser inserida.

Para inserirmos uma linha na tabela REGIÃO, o comando poderia ser:

Adicionando uma linha na tabela.

Outra forma de comando é omitindo as colunas como neste comando:

Adicionando uma linha na tabela.

Nesse caso, o primeiro valor vai para a primeira coluna da tabela, ID_REGIAO, e o segundo valor para a segunda coluna, NOME. Após a inserção, teremos:

Adicionando linhas na tabela.

Observação: O Select anterior mostra o conteúdo da tabela.

Atualizando linhas nas tabelas

O comando UPDATE permite que atualizemos dados já existentes nas tabelas. Sua sintaxe é:



Em que as cláusulas do comando UPDATE são:

- nome_tabela – é o nome da tabela a ser atualizada.
- coluna – é a coluna que queremos alterar.
- expressão – é o novo valor associado à coluna alterada.
- condição – é a condição que deverá satisfazer a coluna que será alterada.

Por exemplo, para alterar o nome da região 1 para EXEMPL01, o comando seria:

Alterando linha na tabela.

Se você omitisse o WHERE as duas linhas seriam atualizadas. Após o comando, a tabela ficaria da seguinte forma:

Alterando linha na tabela.

Eliminando linhas nas tabelas

O comando DELETE é utilizado para excluirmos linhas em uma tabela e tem como sintaxe:

POSTGRESQL



Em que as cláusulas do comando DELETE são:

- nome_tabela – é o nome da tabela a ser deletada.
- condição – é a condição que deverá satisfazer a coluna que serão deletadas.

Para apagar todas as linhas de REGIAO, o comando é o seguinte:

Excluindo linhas na tabela.

Com isso, como não colocamos cláusula WHERE, todas as linhas serão apagadas.

Eliminando tabelas

Para eliminar uma tabela, temos o Comando Drop, cuja sintaxe é:

POSTGRESQL



Para podermos apagar as tabelas referenciadas em chaves estrangeiras, temos que apagar primeiro as tabelas que as referenciam.

No caso de nosso banco, a sequência de comandos seria:

Excluindo tabelas.

Controlando transações

Uma transação é um conjunto de operações no banco de dados. Para efetivarmos as operações realizadas pelo comando, devemos utilizar o comando de COMMIT. Por outro lado, se desejamos desfazer, devemos comandar ROLLBACK.

O pgAdmin4 vem configurado para AUTO COMMIT, ou seja, efetivar imediatamente os comandos após a sua execução.

Configuração de AUTO COMMIT.

Se você desmarcar a opção mostrada na imagem, terá que comandar COMMIT após as operações de CREATE, ALTER, DROP, INSERT, UPDATE e DELETE.

Atividade 3

A linguagem SQL fornece uma série de comandos para permitir a criação de objetos e manipulação de dados. O comando que permite que façamos alterações nas linhas de uma tabela é o

- A UPDATE.
- B ALTER.
- C UPGRADE.
- D DROP.
- E ALTER DATA.

Parabéns! A alternativa A está correta.

Os comandos para manipulação de dados que compõem a DML são:

Insert – para inserir linhas das tabelas.

Delete – para eliminar linhas das tabelas.

Update – para alterar a linhas da tabela.

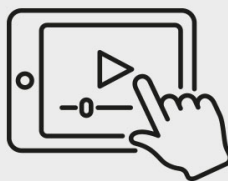
Criação de um banco de dados na prática

Vamos agora criar o banco de dados EMPRESA, que utilizaremos no restante de nosso estudo.

O vídeo mostrará a criação passo a passo. Você pode baixar o script da criação das tabelas e inserção das linhas do banco de dados da empresa no campo Preparação no início deste conteúdo.

Neste vídeo, vamos praticar a criação de um banco de dados no PostgreSQL utilizando a ferramenta pgAdmin4. Assista!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro da prática

Para criar o banco de dados da empresa, confira os seguintes passos!

1. Abra o pgAdmin4 e faça conexão no servidor.
2. Crie um banco de dados chamado EMPRESA.
3. Abra uma janela de consulta.
4. Carregue o script da empresa na janela de consulta.
5. Execute o script.
6. Valide a criação das tabelas na aba Tabelas do banco da empresa.

Observe, a seguir, o exemplo de tabela gerado.

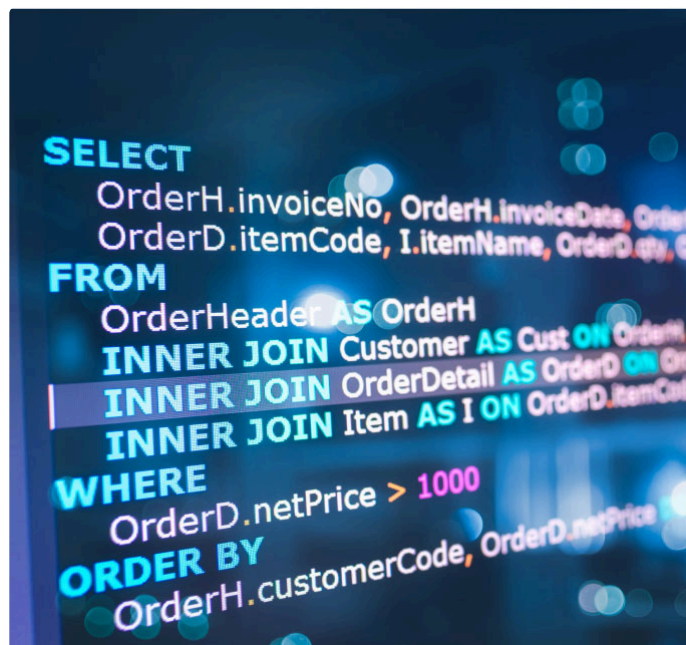
Tabelas do banco de dados EMPRESA.

Faça você mesmo!

Para fixar os conteúdos referentes ao controle de transação, execute o exercício descrito no arquivo controle de transação, que se encontra no material disponibilizado no campo Preparação no início deste conteúdo (exercmod1nc4.docx).

Chave de resposta ▾

O arquivo da solução do exercício (solexercmod1nc4.txt) também se encontra no material baixado.



2 – Comandos LMD com uma tabela

Ao final deste módulo, você será capaz de consultar dados em uma tabela.

SELECT básico

Vamos iniciar o nosso estudo do comando SELECT. Em nossos exemplos, utilizaremos o banco de dados da empresa. Observe!

Modelo de dados do estudo de caso.

As tabelas possuem os seguintes dados:

	id_regiao numeric (7)	nome character varying (40)
1	1	Norte
2	2	Sul

Tabela REGIAO

	id numeric (7)	nome character varying (40)	id_regiao numeric (7)
1	10	Administrativo	1
2	20	Vendas	1
3	30	Compras	2

Tabela DEPARTAMENTO

	id numeric (7)	ult_nome character varying (20)	prim_nome character varying (20)	cargo character varying (30)	salario numeric (7,2)	dt_admissao date	cpf character (11)	id_depto numeric (7)	id_gerente numeric (7)
1	1	Velasques	Carmen	Presidente	29500.00	2009-05-05	34567890125	10	[null]
2	2	Neves	Lauro	Diretor de Compras	19500.00	2009-03-03	23456789012	30	1
3	3	Nogueira	Ermene	Diretor de Vendas	18000.00	2010-04-07	34567890123	20	1
4	4	Queiroz	Mark	Gerente de Compras	8000.00	2010-11-11	12345432123	30	2
5	5	Rodrigues	Alberto	Vendedor	4000.00	2008-10-10	87965432123	20	3
6	6	Ugarte	Mariene	Vendedor	3500.00	2009-03-03	87654345678	20	3

Tabela EMPREGADO

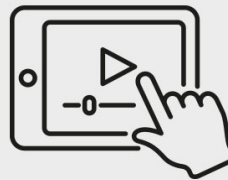
	id numeric (7)	nome character varying (40)	vendedor numeric (7)
1	110	Ponto Quente	5
2	120	Casa Supimpa	6
3	130	Coisas e Tralhas	5
4	140	Casa Desconto	[null]

Tabela CLIENTE

Se desejar, você pode encontrar no arquivo sqlmod2nc1, disponibilizado no material para download no campo **Preparação**, os comandos utilizados nos exemplos a seguir.

Assista a este vídeo e entenda como implementar consultas em tabelas utilizando comandos SELECT básico.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Retornando colunas de uma tabela

O comando SQL que nos permite recuperar dados de uma tabela é o SELECT e sua sintaxe básica é:

POSTGRESQL



Em que as palavras-chave do comando SELECT são:

- nome_tabela – é o nome da tabela que contém os dados a serem recuperados.
- nome_coln – é o nome de uma coluna a ser recuperada.
- * (asterisco) – recupera todas as colunas da tabela.

Vamos listar o conteúdo da tabela EMPREGADO abrindo uma nova janela de consulta, observe!

Comando SELECT.

No Select, você pode também selecionar apenas algumas colunas e determinar a ordem em que retornam. Execute o seguinte comando:

Comando SELECT.

No comando acima, são selecionadas apenas três colunas e todas as linhas da tabela EMPREGADO.

Escrevendo expressões aritméticas em comandos SELECT

Em uma expressão, podemos especificar não apenas uma coluna, mas um dado derivado de uma ou mais colunas, por exemplo, na tabela EMPREGADO temos o salário mensal. Se desejarmos calcular o salário anual entendido como 12 salários somado ao décimo terceiro mais um terço de férias, precedido pelo ID e PRIM_NOME do empregado, podemos escrever o seguinte comando:

Comando SELECT.

Escrevendo expressões de concatenação

O operador de concatenação é o "||". Se desejássemos retornar o nome completo dos empregados como uma única cadeia de caracteres, o comando seria:

Comando Select.

Observou que os nomes estão grudados? Isso acontece porque depois do PRIM_NOME ou antes do ULT_NOME não existe espaço em branco armazenado na coluna. Como resolver isso?

Basta concatenar as colunas com um espaço em branco entre eles.

Comando SELECT.

Criando alias de coluna

Quando são utilizadas expressões, o cabeçalho da coluna, normalmente, fica sem significado. Para isso, existe o alias de coluna. Você deve

colocar AS e, a seguir, entre aspas o título da coluna se tiver espaço em branco ou sem aspas se for uma única palavra, sendo o AS opcional.

Comando SELECT.

Comando SELECT.

Utilizando SELECT sem from

A cláusula SELECT, além de permitir a realização da projeção das colunas da tabela, pode ser utilizada para exibir resultados de operações aritméticas, retorno de funções ou textos.

Vejamos um exemplo!

Comando SELECT.

Note que:

1. O comando primeiro exibe um texto (ALO), a seguir o resultado de uma operação aritmética (9+5) e finalmente retorna o valor da data da data/hora do sistema (FUNÇÃO NOW()).
2. Não existe cláusula FROM, pois desejamos retornar apenas uma linha com os valores.

Atividade 1

Veja a seguir a tabela Empregado em um SGBD PostgreSQL:

Tabela Empregado.

Sobre essa tabela, foi executado um comando que produziu o seguinte resultado:

Resultado de consulta na tabela Empregado.

O comando que produziu esse resultado foi:

- A `SELECT ID, PRIM_NOME + ' ' + ULT_NOME AS "NOME COMPLETO" FROM EMPREGADO`
- B `SELECT ID, PRIM_NOME || ' ' || ULT_NOME AS 'Nome Completo' FROM EMPREGADO`
- C `SELECT ID, PRIM_NOME || ' ' || ULT_NOME AS "Nome Completo" FROM EMPREGADO`
- D `SELECT ID, PRIM_NOME || ' ' || ULT_NOME AS "NOME COMPLETO" FROM EMPREGADO`
- E `SELECT ID, PRIM_NOME + ' ' + ULT_NOME AS "Nome Completo" FROM EMPREGADO`

Parabéns! A alternativa C está correta.

O operador de concatenação do PostgreSQL é o `||` não o `+`, o alias tem que vir entre aspas (`"`), não entre apostrofes (`'`) e está grafado com a primeira letra de cada palavra em maiúsculas, não todas as letras.

SELECT com WHERE

A cláusula `Where` estabelece uma condição que a linha deverá obedecer para que faça parte do conjunto resposta da consulta.

Para os exemplos, continuaremos a utilizar o banco de dados da empresa e novamente sugerimos que você execute os comandos de exemplo no PostgreSQL.

Se desejar, você pode encontrar no arquivo `sqlmod2nc2` os comandos utilizados nos exemplos.

Entenda neste vídeo como implementar consultas em tabelas utilizando comandos `SELECT COM WHERE`.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Cláusula WHERE

É uma cláusula que estabelece uma condição que a linha deverá obedecer para que faça parte do conjunto resposta da consulta. Vamos retornar as linhas dos empregados com ID > 3.

Comando SELECT com WHERE.

Repare que, das seis linhas que a tabela possui, empregados com ID de 1 até 6 somente retornam as três que possuem ID maior que 3.

Na construção das condições, você pode utilizar os seguintes operadores relacionais:

- = Igual a
- <> Diferente
- > Maior que
- < Menor que
- >= Maior ou igual
- <= Menor ou igual

Um cuidado que você deve tomar é com o tipo de dado que está utilizando para filtrar. E se fosse um texto? Vejamos!

Tabela CLIENTE.

Você deseja o ID do cliente Ponto Quente, cujo valor é 110. Confira qual seria o comando!

Comando SELECT com WHERE.

Por que não voltou o ID do cliente já que existe esse nome na tabela? Alguns SGBD são “case sensitive” para os dados, ou seja, uma letra maiúscula é diferente da letra minúscula.

Comando SELECT com WHERE.

Nesse caso, você deve padronizar a consulta utilizando uma função que leve o valor existente no banco de dados para maiúsculo ou para minúsculo antes de fazer a comparação.

Veja o exemplo!

Comando SELECT com WHERE.

Ao trabalharmos com datas, devemos colocá-las entre aspas simples, no formato 'dd/mm/aaaa', em que dd é o dia em dois dígitos, mm o mês em dois dígitos e aaaa o ano em quatro dígitos. Por exemplo:

Comando SELECT com WHERE.

Consultando dados com várias condições

Você pode especificar critérios complexos combinando várias condições de pesquisa utilizando operadores lógicos.

Vamos supor que você deseja mostrar os empregados que tenham sido contratados após primeiro de janeiro de 2010 e que tenham salário maior que 10000.

Confira o comando!

Comando SELECT com WHERE.

Repare que a utilização do AND obriga que as duas condições sejam verdadeiras para que a linha retorne; se tivéssemos utilizado OR, bastaria uma ser verdadeira.

Comando SELECT com WHERE.

Operadores da linguagem SQL

Vamos conferir alguns operadores importantes da linguagem.

Operador IN

Permite comparar o valor da coluna com uma lista de valores e retorna verdadeiro se em determinada linha o valor da coluna for igual a um dos valores da lista.

Para listarmos os empregados que trabalham no departamento 10 ou no 30, o comando seria:

Comando SELECT com WHERE.

Agora, para ver os empregados que **não** trabalham nesses departamentos, basta acrescentar NOT ao comando. Observe!

Comando SELECT com WHERE.

Operador BETWEEN

O operador BETWEEN AND permite verificar se o valor de um campo está contido em uma faixa de valores. Por exemplo, para retornar o ID, ULT_NOME E SALARIO dos empregados com salários entre 8000 e 19500 inclusive.

O comando seria:

Comando SELECT com WHERE.

Repare que retornam os empregados com salário de 8000 e de 19500 – isso mostra que o BETWEEN cria um intervalo fechado.

Para fazer a condição inversa, basta utilizar NOT BETWEEN.

Comando SELECT com WHERE.

Operador LIKE

É utilizado para fazer casamento de padrão, ou seja, procurar um conjunto de caracteres que exista em uma cadeia de caracteres. Esse operador necessita do uso de caracteres curingas, entenda melhor!

%

Curinga para representar uma quantidade arbitraria de caracteres (inclusive nenhum).

—

Curinga para indicar a existência obrigatória de um caractere naquela posição.

Caso se deseje saber o id e ULT_NOME dos empregados com a letra E na segunda posição do sobrenome, temos o seguinte comando:

Comando SELECT com WHERE.

Para os que não possuem, basta acrescentar NOT.

Comando SELECT com WHERE.

Operador IS NULL

Determina se no campo existe valor ou se não existe (campo nulo).
Considere a tabela CLIENTE.

Tabela CLIENTE.

Se você deseja retornar todos os dados dos CLIENTES que não são atendidos por um vendedor, o comando será:

Comando SELECT com WHERE.

Caso contrário, você deve utilizar IS NOT NULL.

Comando SELECT com WHERE.

Atividade 2

Vejamos a seguir o conteúdo de algumas colunas da tabela EMPREGADO em um SGBD PostgreSQL.

Colunas da tabela Empregado.

Se fosse emitido o seguinte comando,

Comando SQL.

Retornariam apenas os empregados de ID:

A 1

B 1 e 2

C 2 e 5

D 1, 2 e 6

E 1, 2 e 5

Parabéns! A alternativa E está correta.

O comando solicita ao empregado cuja penúltima letra do ult_nome seja E.

Comando SQL.

Aplicação prática SELECT básico + WHERE

Vamos agora ver exemplos de uso dos comandos vistos até agora. Para isso, iremos utilizar o banco de dados do museu, cujo modelo lógico é o seguinte:

Modelo lógico do banco de dados Museu.

Você pode baixar o script da criação das tabelas e inserção das linhas do banco de dados do museu, arquivo `museu.sql`, disponibilizado no campo Preparação.

Neste vídeo, vamos demonstrar como implementar consultas em tabelas utilizando comandos SELECT com WHERE. Não perca!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro da prática

Para criar o banco de dados do museu, confira os seguintes passos:

1. Abra o pgAdmin4 e faça conexão no servidor.
2. Crie um banco de dados chamado MUSEU.
3. Abra uma janela de consulta.
4. Carregue o script do museu na janela de consulta.
5. Execute o script.
6. Valide a criação das tabelas na aba tabelas do banco do museu.

Veja a tabela:

Tabelas do banco Museu.

Escreva os seguintes comandos de consulta:

- Listar o conteúdo da tabela de autores.
- Listar o nome e nacionalidade de todos os autores.
- Listar CPF, nome, salário e salário anual dos funcionários.
- Listar o nome do autor concatenado com a sua nacionalidade.
- Listar o nome do autor concatenado com a sua nacionalidade com o alias de coluna nome e nacionalidade.
- Listar todos os dados dos autores com código maior que 105.
- Listar todos os dados dos autores com código maior que 105 e de nacionalidade francesa.
- Listar todos os dados dos autores franceses ou brasileiros.
- Listar todos os dados dos autores com código entre 104 e 107 inclusive.
- Listar todos os dados dos autores com dali no nome.
- Listar todos os dados dos salões cuja área é nula.

Você pode baixar os comandos que resolvem as consultas no arquivo `sqlmod2nc3` disponibilizado no campo Preparação.

Faça você mesmo!

Vamos agora executar uma atividade prática utilizando o banco de dados da escola, cujo modelo lógico é apresentado na imagem. Baixe o script `escola.sql`, referente à criação das tabelas e inserção das linhas do banco de dados, crie o banco de dados da escola, faça conexão e realize as consultas solicitadas no exercício descrito no arquivo `exercmod2nc3`, que se encontram no material disponibilizado no campo Preparação no início deste conteúdo.

Modelo lógico do banco de dados Escola.

Chave de resposta ▾

O arquivo da solução do exercício (`solexermod2nc3.txt`) também se encontra no material baixado.

Funções de agregação e GROUP BY + ORDER BY

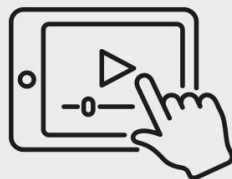
Agora veremos como gerar dados agregados utilizando funções de grupo, eliminando linhas duplicadas e ordenando a saída da consulta.

Para os exemplos, continuaremos a utilizar o banco de dados da empresa e novamente sugerimos que você execute os comandos de exemplo no PostgreSQL.

Se desejar, você pode encontrar no arquivo `sqlmod2nc4` os comandos utilizados nos exemplos.

Entenda, por meio deste vídeo, como implementar consultas utilizando funções de agregação, `GROUP BY` e `ORDER BY`.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Eliminando valores duplicados (`DISTINCT`)

Analise o conteúdo da tabela `EMPREGADO`.

Tabela `EMPREGADO`.

Observe que na tabela empregados Rodrigues e Ugarte possuem o mesmo cargo. E se desejássemos ver os diferentes cargos que temos?

Considere o seguinte comando:

Comando `SELECT`.

Apesar de termos recuperado todos os cargos, os repetidos aparecem várias vezes.

Nesse caso, podemos acrescentar a cláusula `DISTINCT`, retornando apenas uma vez cada cargo.

Comando `SELECT` com `DISTINCT`.

Observe agora o seguinte comando:

Comando `SELECT` com `DISTINCT`.

Note que voltaram os dois vendedores. Isso ocorreu devido ao fato de `DISTINCT` filtrar as linhas diferentes, não os valores da coluna; como os dois vendedores possuem nomes diferentes, as linhas são distintas e, portanto, as duas retornam.

Funções de grupo

Uma função de grupo atua em uma instância da tabela, ou seja, no conjunto de suas linhas.

As funções agregam os dados a partir de todas as linhas da tabela ou de grupos em que as linhas possam ser enquadradas. Vejamos as funções de grupo!

AVG(coluna)

MÉDIA dos valores da coluna solicitada.

MAX(coluna)

MAIOR valor existente na coluna solicitada.

MIN(coluna)

MENOR valor existente na coluna solicitada.

SUM(coluna)

SOMATÓRIO dos valores da coluna solicitada.

COUNT(*)

CONTAGEM das linhas que a consulta retornaria.

Considere os dados da tabela Empregado:

Tabela EMPREGADO.

Para obtermos as agregações das funções para o salário do empregado, o comando deve ser:

Funções de agregação.

Agora, você deseja listar o maior e o menor valor de salário de um empregado; os sobrenomes que aparecem como último e como primeiro na ordem alfabética crescente; e as datas de contratação do empregado mais antigo e mais novo. Para isso, basta comandar:

Funções de agregação.

Criando grupo nas tabelas

Podemos agrupar os dados de nossa tabela utilizando a cláusula GROUP BY. Por exemplo, para listar a quantidade de empregados, a

média salarial e o valor da folha salarial de cada departamento, o comando seria:

Cláusula GROUP BY.

Observe o seguinte:

- Na cláusula GROUP BY, você deve colocar a coluna cujos valores serão utilizados para criar os grupos, sendo que pertencem ao mesmo grupo todas as linhas que possuem valores iguais na coluna.
- Você também deve colocar na cláusula SELECT na coluna ou colunas utilizadas para criar os grupos, permitindo que se saiba a qual grupo pertencem o valor gerado pela função de grupo.

GROUP BY permite que você crie grupos definidos por mais de uma coluna, ou seja, a agregação não será por uma coluna, mas por um conjunto de colunas.

Para recuperar a quantidade de empregados que temos em cada cargo em cada departamento, o comando seria:

Cláusula GROUP BY.

Filtrando dados agrupados

Ao se utilizar a cláusula GROUP BY, é possível filtrar a seleção de dados antes deles serem agrupados ou/e após o agrupamento.

O momento de realizar essa restrição vai depender se deseja eliminar linhas da tabela antes de aplicar as funções de grupo, criando uma condição na cláusula WHERE, ou se o que você quer é eliminar GRUPOS que não atendam a uma determinada condição, utilizando, neste caso, a cláusula HAVING.

Por exemplo, para filtrar as linhas antes de agrupar, vamos contar a quantidade de empregados, o salário médio e a soma dos salários dos empregados do departamento 20. Observe!

Filtrando resultados de funções de agregação.

Além disso, podemos filtrar os grupos, por exemplo, retornar à quantidade de empregados, média salarial e soma dos salários para os departamentos que possuem mais de um empregado. Veja!

Filtrando resultados com a cláusula HAVING.

Ordenando consultas

Muitas vezes, desejamos que as linhas sejam ordenadas; para isso, temos a cláusula ORDER BY.

O resultado de uma consulta pode ser ordenado pelo valor de uma ou mais colunas de forma crescente, que é o padrão, ou decrescente, utilizando DESC.

Por exemplo, para retornar os dados dos empregados ordenados na ordem alfabética decrescente de ID_DEPTO e crescente de salário, o comando é:

Ordenando com a cláusula ORDER BY.

Outra forma de comandarmos a ordenação é pela posição da coluna no resultado. Com isso o, comando anterior poderia ser escrito da seguinte forma:

Ordenando com a cláusula ORDER BY.

Importante: A cláusula ORDER BY será sempre a última de um comando SQL.

Atividade 3

A imagem a seguir mostra o conteúdo de algumas colunas da tabela EMPREGADO em um SGBD PostgreSQL.

Colunas da tabela Empregado.

Observe o comando:

Comando SQL.

Se fosse emitido o comando anterior, qual seria o retorno?

- A Marlene
- B Ugarte
- C Velasques
- D Carmen
- E Neves

Parabéns! A alternativa A está correta.

O comando retorna o prim_nome que aparece por último na ordem alfabética crescente.

Resultado de comando SQL.

Aplicação prática de funções de agregação e GROUP BY + ORDER BY

Vamos agora ver exemplos de uso dos comandos vistos até agora. Para isso, iremos utilizar o banco de dados do museu, cujo modelo lógico é o seguinte:

Modelo lógico do banco de dados museu.

Confira neste vídeo como implementar consultas utilizando comandos com funções de agregação, GROUP BY e ORDER BY.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade, confira os seguintes passos:

1. Abra o pgAdmin4 e faça conexão no servidor.
2. Abra uma janela de consulta banco do museu.
3. Escreva os comandos de consulta listados a seguir.

Comandos de consulta:

- Listar a área média com o alias área média, e número de salões com o alias total de salões.
- Listar o estilo e o total de pinturas do estilo com o alias 'quantidade de pinturas'.
- Listar o estilo e o total de pinturas do estilo com o alias 'quantidade de pinturas' para os estilos diferentes de cubista.
- Listar o estilo e o total de pinturas do estilo com o alias 'quantidade de pinturas' para os estilos diferentes de cubista e com pelo menos.
- Listar os diferentes estilos de pintura.
- Listar todos os dados dos autores em ordem descendente de nacionalidade.
- Listar todos os dados dos autores em ordem ascendente de nacionalidade e descendente de código.
- Repetir o comando anterior ordenando pela posição da coluna.

Você pode encontrar os comandos que resolvem as consultas no arquivo sqlmod2nc5.

Faça você mesmo!

Vamos agora executar uma atividade prática utilizando o banco de dados da escola. Observe o modelo lógico! Para realizar o exercício,

faça conexão e realize as consultas solicitadas no exercício descrito no arquivo exercmod2nc5.

Modelo lógico do banco de dados escola.

Chave de resposta ▾

O arquivo da solução do exercício (solexermod2nc5.txt) também se encontra no material baixado.



3 – Comandos LMD com duas ou mais tabelas

Ao final deste módulo, você será capaz de consultar dados em mais de uma tabela.

Subconsultas

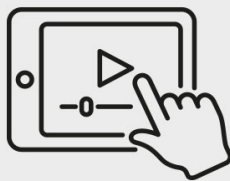
Para obter a média dos valores segurados e utilizá-la para filtrar linhas de uma tabela, temos as subconsultas.

Continuaremos a utilizar o banco de dados da empresa e novamente sugerimos que você execute os comandos de exemplo no PostgreSQL.

Se desejar, você pode encontrar no arquivo `sqlmod3nc1` os comandos utilizados nos exemplos.

Neste vídeo, vamos explicar como implementar consultas utilizando subconsultas. Assista!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Subconsultas aninhadas

São consultas secundárias dentro de outras utilizadas para gerar valores intermediários necessários na execução da consulta principal.

Por exemplo, você deseja recuperar todos os dados dos empregados cujo salário seja maior que o salário médio da empresa. Como não existe o salário médio armazenado em nenhuma tabela, podemos utilizar uma subconsulta para calculá-lo e em seguida utilizar o valor obtido para filtrar as linhas de empregado na cláusula `WHERE`.

Exemplo de Subconsulta.

Observe que existe um comando `SELECT` (consulta inicial) e um outro comando `SELECT` dentro desse primeiro (interno), o que caracteriza a subconsulta aninhada.

O seu modo de execução é o seguinte: o segundo `SELECT`, interno, é executado primeiramente, gerando a informação que será utilizada pelo `SELECT` principal.

Exemplo de subconsulta.

O valor retornado (13750) é então utilizado pela consulta principal, sendo executada como se tivesse sido escrita da seguinte forma:

Exemplo de Subconsulta.

Subconsulta simples

São aquelas que retornam apenas uma linha, como acontece, por exemplo, ao se utilizar uma função de grupo sem cláusula `GROUP BY`, como no caso da `SUBCONSULTA`, que calculou a média salarial.

Nesse tipo de subconsulta, podem ser utilizados os seguintes operadores na comparação:

- = Igual a
- <> Diferente
- > Maior que
- < Menor que
- >= Maior ou igual
- <= Menor ou igual

Subconsultas multilinhas

São aquelas que retornam mais de uma linha, o que nos obriga a utilizar o operador IN ou NOT IN, pois somente esses operadores conseguem lidar com um conjunto de valores.

Para maior clareza, confira os exemplos a seguir.

A tabela de DEPARTAMENTOS tem o seguinte conteúdo:

Tabela DEPARTAMENTOS.

Temos dois departamentos da região 1 e um da região 2. Se quisermos listar o id, ULT_NOME e ID_DEPTO dos empregados do departamento da região 1, como teremos dois departamentos, não podemos utilizar "=", temos que usar o operador IN. Observe!

Exemplo de subconsulta.

Nossa sugestão é que você substitua o IN por "=" no comando anterior e veja o que acontece.

Atenção!

Um cuidado que temos que tomar, quando retornamos várias linhas, é que se uma linha for nula o teste NOT IN pode falhar.

A tabela CLIENTE possui o seguinte conteúdo:

Registros da tabela CLIENTE.

Note que o cliente 140 não possui vendedor que o atende, o campo é nulo.

Caso queira listar o ID, ULT_NOME e CARGO dos empregados que atendem clientes, o comando será:

Exemplo de subconsulta.

No entanto, se você tentar ver os que não atendem, não retorna nenhuma linha:

Exemplo de subconsulta.

Como já foi destacado, existe um nulo no retorno da subconsulta. Quando se usa IN, não ocorre nenhum problema, mas quando é utilizado NOT IN a comparação com NULO resulta NULO.

Dessa forma, **nunca** permita o retorno de linha nula em uma subconsulta!

Exemplo de subconsulta.

Subconsulta correlata

Além das subconsultas aninhadas, existe outro tipo denominada correlata, que, para poder ser executada, necessita de valores da consulta principal, por isso, deve ser executada uma vez para cada linha da consulta externa.

Por exemplo, considere que você deseja retornar o ID, ULT_NOME, SALARIO, ID_DEPTO dos empregados que ganham igual ou mais que o salário médio de seu departamento. Para resolver isso, é necessário que:

1. A subconsulta acesse na consulta principal qual o departamento do empregado cuja linha está sendo processada naquele momento.
2. Calcule a média do valor para aquele departamento.
3. Retorne a média para consulta principal.
4. Na consulta principal, compare a média com o salário do empregado da linha que está sendo processada.
5. Retorne os dados do empregado se o valor segurado for menor que a média.

Para se obter a situação descrita, a subconsulta deve referenciar dados da consulta principal e, para isso, você deve dar um apelido para a tabela da consulta principal e outro para a tabela da subconsulta, e na subconsulta, "chamar" os dados da tabela principal conforme mostra o comando:

Exemplo de subconsulta correlata.

Observe no comando que a tabela externa foi apelidada de E a interna de I e no Where foi utilizado o valor da tabela externa para filtrar as linhas da subconsulta.

Exemplo de subconsulta correlata.

Teste de existência

Uma forma mais elegante de realizar a consulta para obter os vendedores, sem se preocupar com o retorno nulo, é utilizar o operador EXISTS. Esse operador exige o uso de uma subconsulta correlata que verifica se retornam ou não linhas na subconsulta.

Caso VERDADEIRO

Se o retorno do teste resulta em verdadeiro, a linha testada é colocada no resultado.

Caso FALSO

Se o retorno do teste resulta em falso a linha testada é descartada da resposta.

Empregados que atendem clientes:

Exemplo de uso de EXISTS.

Empregados que não atendem clientes:

Exemplo de uso de EXISTS.

Atividade 1

Mostramos a seguir o conteúdo de algumas colunas da tabela EMPREGADO em um SGBD PostgreSQL.

Colunas da tabela EMPREGADO.

Observe o comando:

Comando SQL.

Se fosse emitido o comando anterior, retornariam apenas os empregados de ID:

A 2 e 4

B 1

C 3 e 5

D 1, 2, 5 e 6

E 2, 5 e 6

Parabéns! A alternativa E está correta.

O comando solicita ao empregado cujo ID_DEPTO seja maior que o ID_DEPTO do empregado cujo cargo tenha a palavra VENDAS no final, que corresponde ao empregado Nogueira do departamento 20, portanto, voltam os empregados do departamento 30.

Resultado de consulta com subconsulta.

Operadores de conjunto

Algumas vezes é necessário fazer a chamada JUNÇÃO VERTICAL, que nada mais é do que retornar dados de mais de uma tabela em uma

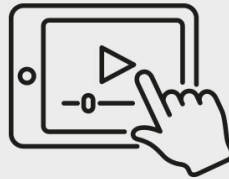
mesma coluna. Isso pode ser feito utilizando os operadores de conjunto UNION, INTERSECT e EXCEPT.

Para os exemplos, continuaremos a utilizar o banco de dados da empresa e novamente sugerimos que você execute os comandos de exemplo no PostgreSQL.

Se desejar, você pode encontrar no arquivo sqlmod3nc2 os comandos utilizados nos exemplos.

Assista a este vídeo e compreenda como implementar consultas utilizando comandos com operadores de conjunto.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Utilizando operadores de conjunto

Toda tabela pode ser considerada um conjunto de valores e os operadores de conjunto nos permitem realizar as operações de:



UNIÃO

Operador UNION

Realiza a operação, ou seja, retorna todas as linhas do primeiro conjunto acrescidas de todas as linhas do segundo conjunto, eliminando as duplicadas.



INTERSEÇÃO

Operador INTERSECT

Realiza a operação, ou seja, retorna as linhas comuns aos dois conjuntos.



DIFERENÇA

Operador EXCEPT

Realiza a operação, ou seja, retorna as linhas que existem no primeiro conjunto e não existem no segundo.

Para gerar os conjuntos a serem operados, são utilizadas subconsultas para gerar tabelas em memória, mas para que o comando funcione, é necessário que as tabelas geradas sejam compatíveis.



Mas o que significa tabelas compatíveis?

Resposta

Duas ou mais tabelas são compatíveis para fazer operações de conjunto se possuírem o mesmo esquema, ou seja, o mesmo número de colunas, com o mesmo tipo, na mesma ordem.

Observe a seguir.

Compatibilidade entre tabelas.

As tabelas Empregados e Produtos são compatíveis, já que possuem três colunas, sendo a primeira e a segunda numéricas e a terceira alfanumérica. Já a tabela Animais não é compatível com as outras duas, embora possua três colunas, sendo duas numéricas e uma alfanumérica, as quais não se encontram na mesma ordem, pois a alfanumérica é a primeira e as numéricas as duas últimas.

Por definição, conforme você deve ter aprendido na matemática, as operações de conjunto eliminam valores duplicados. Isso quer dizer que se duas tabelas possuírem linhas iguais, apenas uma linha irá retornar ao resultado. Observe!

Compatibilidade entre tabelas.

Ao se comparar a primeira linha da tabela Empregado e cada uma das linhas da tabela Acionista, observa-se que não existe nenhuma linha igual na tabela de Acionista. Isso porque, apesar de existir um acionista com o mesmo ID (primeira linha, id 101), o nome é diferente, portanto, essas linhas não são iguais.

O mesmo ocorre em relação à terceira linha de Empregado, pois a segunda linha de Acionista possui o mesmo valor para nome (Marco Aurélio), porém com valor de Id diferente. Já a segunda linha da tabela Empregado é exatamente igual à última linha de Acionista, já que possuem o mesmo nome e o mesmo ID. Caso a duas fizessem parte do conjunto resposta de uma operação de conjunto, apenas uma delas apareceria no resultado.

Operador UNION

Realiza a operação de UNIÃO, ou seja, retorna todas as linhas do primeiro conjunto acrescidas de todas as linhas do segundo conjunto, eliminando as duplicadas. Se quisermos, por exemplo, os ids dos empregados que trabalham nos departamentos 20 ou 30, ou que possuem o cargo de vendedor, como ficaria o comando utilizando operadores de conjunto?

Observe a seguir os dados da tabela.

Tabela EMPREGADO.

Note que os empregados de ID 2 a 6 são dos departamentos 20 ou 30 (destacados em azul) e os empregados de ID 5 e 6 são vendedores (destacados em vermelho). Queremos os que estão em um conjunto ou no outro, portanto, os empregados de ID 2, 3,4,5 e 6.

Note ainda que os empregados de ID 5 e 6 aparecem nos dois conjuntos. O comando que resolve o problema é

Operação de União.

Existe uma variação no SQL do comando de UNIÃO que retorna os duplicados é o UNION ALL.

Operação de União.

Agora os ID 5 e 6 aparecem duas vezes.

Operador INTERSECT

Se você desejasse os ids dos empregados que trabalham nos departamentos 20 ou 30 e que possuem o cargo de vendedor, ou seja, que aparecem nos dois conjuntos, utilizaria INTERSECT para obter a interseção dos conjuntos.

Operação de Interseção.

Operador EXCEPT

As duas operações apresentadas (união e interseção) são comutativas, ou seja, tanto faz a primeira subconsulta ser pelo modelo e a segunda pela cor ou o inverso, o resultado será igual. Teste e comprove! O mesmo não acontece na diferença (EXCEPT). Nesse caso, se invertermos a ordem, o resultado mudará.

Para se retornar o id dos empregados que trabalham nos departamentos 20 ou 30 e que não possuem o cargo de vendedor, ou seja, estão no primeiro conjunto e não estão no segundo, o comando seria:

Operação de Diferença.

Já para retornar os vendedores que não são dos departamentos 20 e 30, o comando seria:

Operação de Diferença.

O retorno é vazio devido ao fato de os vendedores serem do departamento 20. Observe!

Operação de Diferença.

Atividade 2

Confira o conteúdo de algumas colunas da tabela empregado em um SGBD PostgreSQL.

Colunas da tabela Empregado.

Observe o comando:

Comando UNION.

Se fosse emitido o comando anterior, retornariam apenas os empregados de ID:

A 2, 4, 5 e 6.

B 2, 3, 4, 5 e 6.

C 1, 3, 5, e 6.

D 1, 3, 4, 5 e 6.

E 1, 2, 3, 4, 5 e 6.

Parabéns! A alternativa D está correta.

No comando emitido, o primeiro conjunto é formado pelos empregados de ID 3,4 e 5, já o segundo conjunto é composto pelos empregados de ID 1,3,5 e 6.

Como foi solicitada a união, o resultado é composto pelos empregados de ID 1,3,4,5 e 6.

Comando Union.

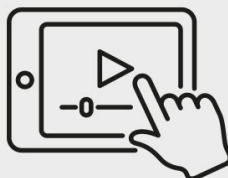
Aplicação prática de subconsultas e operações sobre conjuntos

Vamos agora ver exemplos de uso dos comandos vistos até agora. Para isso, vamos utilizar o banco de dados do museu, cujo modelo lógico é o seguinte:

Modelo de dados do estudo de caso.

Vamos demonstrar, neste vídeo, como implementar consultas utilizando comandos de subconsultas e operações sobre conjuntos. Confira!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade, siga os seguintes passos:

1. Abra o pgAdmin4 e faça conexão no servidor.
2. Abra uma janela de consulta banco do museu.
3. Escreva os comandos de consulta listados a seguir.

Comandos de consulta:

- Listar todos os dados de funcionários cujo salário seja maior que o salário médio de todos os funcionários.
- Listar todos os dados dos salões que possuem obras expostas.
- Listar todos os dados dos salões que não possuem obras expostas.
- Listar os dados dos autores que não possuem obras expostas no museu.
- Listar todos os dados dos salões cujo área seja maior que a área média dos salões do mesmo andar.

- Listar todos os dados dos autores que possuem obras utilizando teste de existência.
- Listar todos os dados dos autores que não possuem obras utilizando teste de existência.
- Listar código dos autores que não possuem obras expostas no museu utilizando operadores de conjunto.
- Listar código dos autores que possuem obras expostas no museu utilizando operadores de conjunto.
- Listar o código dos autores que possuem apenas pinturas no museu utilizando operadores de conjunto.
- Listar o código dos autores que possuem apenas esculturas no museu utilizando operadores de conjunto.
- Listar o código dos artistas que possuem ou apenas esculturas ou apenas pinturas utilizando operadores de conjunto.

Você pode encontrar os comandos que resolvem as consultas no arquivo sqlmod3nc3.

Faça você mesmo!

Vamos agora executar uma atividade prática utilizando o banco de dados da escola. Observe o modelo lógico! Para realizar o exercício, faça conexão e realize as consultas solicitadas no exercício descrito no arquivo exercmod3nc3.

Modelo lógico do banco de dados escola.

Chave de resposta ▾

O arquivo da solução do exercício (solexermod3nc3.txt) também se encontra no material baixado.

Junções internas e externas

Para todos as aplicações práticas, temos basicamente dois tipos de junção:

Junção interior ou interna

Neste tipo retornam as linhas que estão relacionadas a duas tabelas.

Junção exterior ou externa

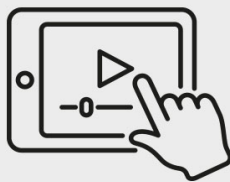
Neste tipo retornam as linhas relacionadas e as não relacionadas de uma ou das duas tabelas.

Para os exemplos, continuaremos a utilizar o banco de dados da empresa e novamente sugerimos que você execute os comandos dos exemplos no PostgreSQL.

Se desejar, você pode encontrar no arquivo `sqlmod3nc4` os comandos utilizados nos exemplos.

Entenda neste vídeo como implementar consultas utilizando comandos com junções internas e externas.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Junção interior

Voltemos à discussão de retornarmos dados de DEPARTAMENTO e REGIAO.

Se você reparar no esquema da tabela DEPARTAMENTO, irá notar a existência de uma chave estrangeira (ID_REGIAO) para a tabela REGIAO, que nos permite saber a região a que o departamento pertence.

Modelo de dados do estudo de caso.

Se você deseja consultar dados dos departamentos e de suas respectivas regiões, por exemplo, você irá utilizar a FK ID_REGIAO para ligar as linhas das duas tabelas, fazendo uma junção.

Como regra geral, podemos definir então que, para fazer junção, também conhecida como JOIN, iremos determinar as colunas em comum nas duas tabelas (PK e FK) e utilizaremos os seus valores para determinar as linhas que irão retornar.

A sintaxe padrão definida pela ANSI para este tipo de junção é:

POSTGRESQL



Em que condição de junção é uma expressão de comparação entre as colunas das duas tabelas envolvidas, normalmente, a chave de primária de uma com a chave estrangeira da outra.

Vamos comandar a junção de DEPARTAMENTO e REGIAO e retornar todas as colunas.

Operação de junção interna.

Observe um detalhe no comando: como as duas tabelas possuem uma coluna ID_REGIAO, temos que realizar a qualificação, ou seja, dizer a qual tabela pertence a coluna ID_REGIAO que estamos referenciando, caso contrário acontecerá um erro de coluna ambígua.

Operação de junção interna.

Para facilitar a qualificação, podemos atribuir um alias à tabela, normalmente uma letra para substituir seu nome, nesse caso, o comando ficaria:

Operação de junção interna.

Junção de mais de duas tabelas

Podemos fazer a junção de mais de duas tabelas. Para isso, basta acrescentarmos uma nova tabela com uma nova cláusula de junção.

A primeira coisa a fazer é identificar quais tabelas podem se juntar.
Analise a modelo do banco:

Modelo de dados do estudo de caso.

Temos uma FK de EMPREGADO para DEPARTAMENTO e uma FK de DEPARTAMENTO para REGIAO, dessa forma, temos que comandar a junção de EMPREGADO com DEPARTAMENTO, que gera uma tabela intermediária em memória, e posteriormente, a junção desse resultado com REGIAO.

Operação de junção interna.

Junção USING

Existe uma outra forma de se escrever um comando de Junção Interior, a Junção Using, mas o que é isso?

Repare que tanto na tabela DEPARTAMENTO quanto na Tabela REGIAO, temos uma coluna chamada ID_REGIAO, FK em Departamento e PK em Região e que seria utilizada na cláusula de junção no comando.

Operação de junção interna.

Quando essa situação ocorre, ao invés de comandarmos INNER JOIN ON, podemos comandar INNER JOIN USING, onde colocamos entre parêntesis a coluna comum a duas tabelas.

Veja o comando a seguir e seu retorno:

Operação de junção interna com USING.

Compare agora esse retorno com o do INNER JOIN ON. As linhas que voltam são as mesmas, mas a coluna ID_REGIAO retorna apenas uma vez no USING.

Autojunção

É uma junção da tabela com ela mesma. Na tabela EMPREGADO, por exemplo, cada empregado está subordinado a outro. A coluna ID_GERENTE indica o código do gerente do empregado. Observe!

Operação de autojunção.

Para mostrarmos uma lista de todos os gerentes, cada um com seus subordinados, podemos comandar:

Operação de autojunção.

Nesse caso, é obrigatório usar um apelido de tabela para distinguir as duas “cópias” da tabela que estão sendo relacionadas: “G” no exemplo representa uma linha da tabela Empregado enquanto Gerente e “S” representa uma linha de um subordinado.

Junção exterior

Este tipo de junção retorna as linhas que estão relacionadas, como no INNER JOIN, e as não relacionadas de uma ou mais tabelas.

Considere, a seguir, a situação de empregados e clientes.

Operação de junção externa.

Para fazermos a junção interior, o comando seria então:

Operação de junção externa.

Note que no resultado voltam apenas os empregados de id 5 e 6, pois estão relacionados como clientes, os demais não aparecem no resultado (id 1,2,3 e 4).

Mas e se desejássemos retornar todos os empregados e para os empregados que são vendedores os dados dos clientes? Comandaríamos então uma junção exterior.

Nesse caso, substituiríamos a expressão INNER pela expressão LEFT OU RIGHT caso a tabela que desejamos retornar esteja do lado esquerdo ou direito do comando. No caso, como empregado está à esquerda, o comando seria:

Operação de junção externa.

Note que as colunas de clientes ficam nulas nas linhas que correspondem aos empregados que não se relacionam com clientes.

Agora, se o comando fosse de RIGHT JOIN, o que aconteceria?

Operação de junção externa.

Retornaríamos todos os clientes e para os que não se relacionam com vendedores (Casa Desconto) as colunas de empregado seriam nulas.

Uma terceira forma de fazer a junção exterior é com FULL JOIN.

Operação de junção externa.

O FULL JOIN retorna todos os relacionados e os não relacionados das duas tabelas, preenchendo com nulo as colunas da outra tabela para as linhas não relacionadas.

Como separamos então os EMPREGADOS que não atendem clientes? Basta acrescentar uma cláusula WHERE que teste se a chave primaria de CLIENTE é nula, pois isso significa que essa linha retornou devido à junção EXTERIOR.

Operação de junção externa.

Note o uso do alias com o * no Select (E.*), pois é uma forma otimizada de pedir para retornar todas as colunas de uma das tabelas, sem ter que especificá-las.

Atividade 3

A seguir mostramos o conteúdo de algumas colunas da tabela empregado em um SGBD PostgreSQL.

Colunas da tabela EMPREGADO.

Confira o comando a seguir:

Comando com junção externa.

Se fosse emitido o comando anterior, retornariam apenas os empregados de GID:

A 1, 2 e 3.

B 2, 3, 4, 5 e 6.

C 3, 5, e 6.

D 3, 4, 5 e 6.

E 4, 5 e 6.

Parabéns! A alternativa E está correta.

É executado um comando de junção exterior no modelo de autojunção, o que faz com que mesmo os empregados que não gerenciam ninguém apareçam como gerentes, ou seja, nas colunas de alias G.

Como na cláusula WHERE eliminamos as linhas que estão com as colunas S.ID nulas, restam apenas os empregados que não são gerentes, ou seja, 4, 5 e 6.

Operação de junção externa.

Aplicação prática de junções

Vamos agora ver exemplos de uso dos comandos vistos até agora. Para isso, iremos utilizar o banco de dados do museu, cujo modelo lógico é o seguinte:

Modelo de dados do estudo de caso.

Assista a este vídeo e aprenda a implementar consultas utilizando comandos com junções.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro da prática

Para executar a atividade, siga os seguintes passos:

1. Abra o pgAdmin4 e faça conexão no servidor.
2. Abra uma janela de consulta banco do museu.
3. Escreva os comandos de consulta listados a seguir.

Comandos de consulta:

- Listar todos os dados dos autores e de suas obras.
- Listar o código, o título e o nome do autor das obras de 1965 a 1975 que estão no salão 36.
- Listar o nome e a nacionalidade dos autores que possuem obras expostas, resolver por junção.
- Listar o nome e a nacionalidade dos autores que não possuem obras expostas, resolver por junção.
- Listar o código e o título das obras do autor Pablo Picasso que se encontram no terceiro andar do museu.
- Listar o nome e a nacionalidade dos autores, o título da obra e o estilo da pintura.
- Listar todos os dados dos salões que não possuem obras expostas.
- Listar o código e o título das obras impressionistas ou cujo material de fabricação é argila, e resolva por junção.
- Gerar uma relação que associa pares de nomes de seguranças diferentes que cuidam dos mesmos salões nos mesmos períodos (mesmos horários de entrada e saída).

Você pode baixar os comandos que resolvem as consultas no arquivo `sqlmod3nc5`.

Faça você mesmo!

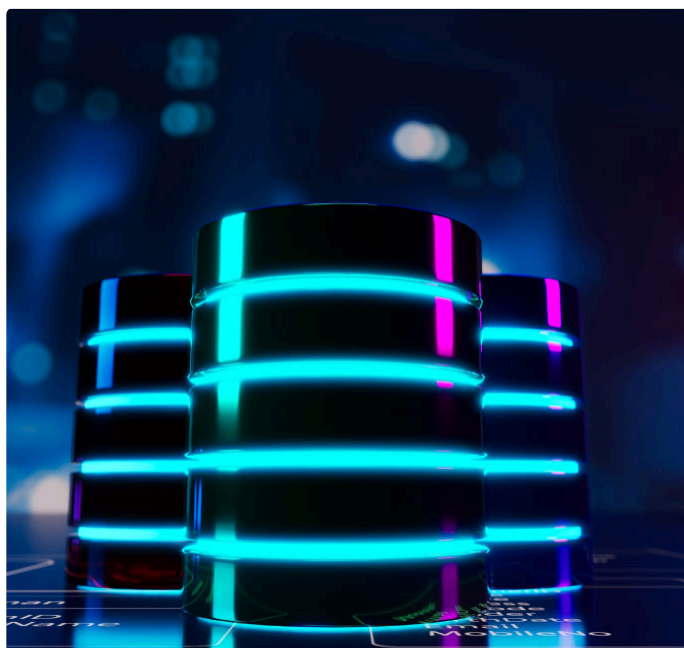
Vamos agora executar uma atividade prática utilizando o banco de dados da escola. Observe o modelo lógico! Para realizar o exercício,

faça conexão e realize as consultas solicitadas no exercício descrito no arquivo exercmod3nc5.

Modelo lógico do banco de dados escola.

Chave de resposta ▾

O arquivo da solução do exercício (solexermod3nc5.txt) também se encontra no material baixado.



4 – Visões e linguagem procedural de banco de dados

Ao final deste módulo, você será capaz de implementar código na utilização de linguagem procedural de banco de dados.

Visões

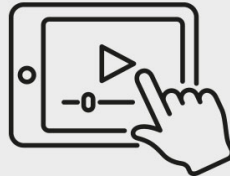
Uma visão (VIEW) é uma consulta previamente definida e que fica armazenada no dicionário de dados, podendo ser acessada de forma similar a uma tabela.

Quando a visão é referenciada em um comando de SELECT, suas linhas e colunas são determinadas dinamicamente, ou seja, a consulta é executada e o resultado apresentado para o usuário, atuando como uma verdadeira tabela virtual.

Uma visão pode permitir, com restrições, que os dados da tabela sejam manipulados em comando de INSERT, UPDATE e DELETE.

Neste vídeo, vamos explicar como criar, consultar e eliminar visões além de conhecer os tipos de VIEWS. Acompanhe!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Criando visões

Vejamos a sintaxe do comando de criação de visões!

POSTGRESQL



Em que cláusulas do comando VIEW:

- nome_view – É o nome da VIEW.
- subquery – É o comando SELECT que originará a VIEW,

Caso você deseje criar uma visão que mostre o ID, o sobrenome e o cargo de um empregado, o comando será:

Criação de visão.

Consultando visões

Para recuperarmos dados através da visão, basta executarmos um comando SELECT, em que o nome da visão fica na cláusula FROM.

Por exemplo, para recuperarmos todas as linhas e colunas da Visão EMP_RESUMO, basta darmos o seguinte comando:

Consulta a uma visão.

Quando fazemos uma consulta a uma VIEW, podemos utilizar todos os recursos que podem ser utilizados em comandos a partir de tabelas.

Eliminando visões

Para eliminar uma visão, basta dar o comando DROP VIEW.

Eliminando visão.

Tipos de views

Existem basicamente dois tipos de views: simples e complexas. Vamos entender!

View Simples

O tipo simples é composto por apenas um SELECT, que utiliza apenas uma tabela e suas colunas são formadas por colunas da tabela original, sem cálculos ou funções.

View complexa

A view complexa é aquela onde há um Join entre tabelas na subquery.

A view complexa é aquela onde há um Join entre tabelas na subquery.

Observações:

- Com uma view simples, será possível executarmos comandos INSERT, UPDATE e DELETE (além do SELECT).
- A manipulação dos dados através de uma view não desabilita as constraints das tabelas as quais eles se referem.
- Cada coluna definida para views deve ter um nome de coluna válido.
- Caso seja uma fórmula, deve possuir um alias.

Veja um exemplo!

Crie a visão CLI_RESUMO com as colunas ID e NOME da tabela cliente.

Criação de visão.

Agora consulte a visão.

Criação de visão.

Note que a nossa visão é simples, então podemos fazer comandos de INSERT, UPDATE e DELETE.

Podemos então inserir um cliente, veja o comando!

Inserindo dado em uma visão.

Se consultarmos a visão, o novo cliente irá aparecer.

Consulta à visão.

E, obviamente, se consultarmos diretamente, a tabela também aparecerá.

Consulta à tabela cliente.

Note que o campo vendedor do novo cliente é nulo, isso ocorre devido ao fato desse campo não ter sido inserido na visão.

Atividade 1

A imagem a seguir mostra o conteúdo de algumas colunas da tabela EMPREGADO em um SGBD PostgreSQL.

Colunas da tabela EMPREGADO.

Considere agora que no banco foi criada a visão V_EMPR com o seguinte comando:

Comando de criação de uma View.

Observe o comando!

Comando SQL.

Se fosse emitido esse comando, retornariam apenas os empregados de ID:

- A 1, 2 e 3.
- B 1, 2, 3 e 4.
- C 2, 3 e 4.
- D 2, 3, 4, 5 e 6.
- E 4, 5 e 6.

Parabéns! A alternativa C está correta.

A visão em sua criação não contemplou o empregado de ID 1, já que não incluiu os empregados do departamento 10.

Resultado de consulta SQL.

Ao fazer o SELECT, retornam apenas os empregados de ID 2, 3 e 4.

Resultado de consulta SQL.

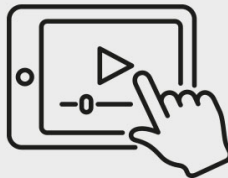
Linguagem procedural – PLPGSQL

O PLPGSQL é uma linguagem de programação completa, incluindo a capacidade de usar loops e outras estruturas de controle. As funções

criadas na linguagem PLPGSQL podem ser chamadas a partir de uma instrução SQL ou como uma ação de um gatilho.

Você sabe como implementar consultas utilizando a linguagem procedural PLPGSQL? Este vídeo é uma ótima oportunidade de compreender, assista!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Estrutura do PLPGSQL

PLPGSQL é uma linguagem estruturada em blocos e cada instrução dentro de um bloco é terminada por um ponto e vírgula.

A sintaxe básica é:

POSTGRESQL



Vamos examinar a estrutura do bloco com mais detalhes!

- Cada bloco possui duas seções: declaração e corpo. A seção de declaração é opcional, enquanto a seção do corpo é obrigatória. Um bloco é finalizado com um ponto e vírgula (;) após a cláusula END.
- Um bloco pode ter um rótulo opcional localizado no início. Você usa o rótulo do bloco quando deseja especificá-lo na EXIT, instrução do corpo do bloco, ou quando deseja qualificar os nomes das variáveis declaradas no bloco.
- A seção de declaração é onde você declara todas as variáveis usadas na seção do corpo. Cada instrução na seção de declaração termina com um ponto e vírgula (;).
- A seção do corpo é onde você coloca o código. Cada instrução na seção do corpo também é encerrada com um ponto e vírgula (;).

Declaração de variáveis

Todas as variáveis utilizadas em um bloco devem ser declaradas na seção de declarações do bloco. As variáveis da linguagem PLPGSQL podem possuir qualquer tipo de dado da linguagem SQL, como integer, varchar e char.

Abaixo seguem alguns exemplos de declaração de variáveis:

Exemplos de declaração de variáveis do PLPGSQL.

A sintaxe geral para declaração de variáveis é:

POSTGRESQL



A cláusula DEFAULT, se for fornecida, especifica o valor inicial atribuído à variável quando o processamento entra no bloco. Se a cláusula DEFAULT não for fornecida, então a variável é inicializada com o valor nulo do SQL.

Instruções básicas

Atribuição

A atribuição de um valor a uma variável é escrita da seguinte maneira:

POSTGRESQL



Veja alguns exemplos:

Exemplos de atribuição de variáveis do PLPGSQL.

SELECT INTO

O resultado de um comando SELECT que retorna várias colunas (mas apenas uma linha) pode ser atribuído a uma variável registro, a uma variável tipo-linha, ou a uma lista de variáveis escalares. É feito através de:

POSTGRESQL



Nesse caso, destino pode ser uma variável registro, uma variável linha ou uma lista separada por vírgulas de variáveis simples e campos de registro/linha.

Vejamos um exemplo!

Exemplo de declaração de variáveis do PLPGSQL

Estruturas de controle

IF-THEN-ELSIF-ELSE

Esta estrutura corresponde ao tradicional IF THEN ELSE, mas fornecendo um método mais conveniente para verificar muitas alternativas em uma instrução. Formalmente, equivale aos comandos IF-THEN-ELSE-IF-THEN aninhados, mas somente necessita de um END IF.

Observe a estrutura básica:

Estrutura de controle IF do PLPGSQL.

Confira um exemplo:

Exemplo de estrutura de controle IF do PLPGSQL.

WHILE

Essa instrução repete uma sequência de instruções enquanto a expressão de condição for avaliada como verdade. A condição é verificada logo antes de cada entrada no corpo do laço.

A sua sintaxe é:

POSTGRESQL



Vejamos um exemplo!

Exemplo de estrutura de controle WHILE do PLPGSQL.

FOR

Cria um laço que interage em um intervalo de valores inteiros. A variável nome é definida automaticamente como sendo do tipo integer, e somente existe dentro do laço. As duas expressões que fornecem o limite inferior e superior do intervalo são avaliadas somente uma vez, ao entrar no laço. Normalmente, o passo da interação é 1, mas quando REVERSE é especificado se torna -1.

Sua sintaxe básica é:

POSTGRESQL



Confira alguns exemplos de FOR:

Exemplo de estrutura de controle For do PLPGSQL.

Procedimentos armazenados

O PostgreSQL 11 introduziu procedimentos armazenados que são subprogramas que possuem suporte às transações.

Para definir um novo procedimento armazenado, você usa o comando CREATE PROCEDURE, cuja sintaxe é:

Estrutura de procedimento armazenado do PLPGSQL.

Nessa sintaxe, confira o passo a passo:

- Especifique o nome do procedimento armazenado.
- Defina os parâmetros para o procedimento armazenado. Um procedimento armazenado pode aceitar zero ou mais parâmetros. Os parâmetros podem possuir os modos IN para entrada e INOUT para entrada e saída.
- Especifique PLPGSQL como a linguagem de procedimento para o procedimento armazenado.
- Use \$\$ no início do código e após o END final e para não ter problemas com cadeia de caracteres use a sintaxe constante de cadeia de caracteres entre aspas para definir o corpo do procedimento armazenado.

Vejamos um exemplo de procedimento armazenado que aumenta o salário dos empregados em um percentual recebido como parâmetro.

Criação de procedimento armazenado.

Vamos agora consultar a tabela EMPREGADO.

Consulta à tabela EMPREGADO.

Para executar o procedimento aumentando os salários em 10%, comande:

Chamada a um procedimento armazenado.

Para ver o resultado, consulte novamente a tabela EMPREGADO.

Consulta à tabela EMPREGADO.

Atividade 2

A seguir, confira o conteúdo de algumas colunas da tabela EMPREGADO em um SGBD PostgreSQL.

Consultas na tabela EMPREGADO.

Sabendo-se que o PLPGSQL é uma linguagem procedural suportada pelo PostgreSQL, o qual permite que sejam escritos procedimentos armazenados, considere o procedimento PROC_QUESTAO criado pelo comando abaixo, em que RAISE NOTICE informa a mensagem a ser exibida por aquele passo da execução.

Exemplo de procedimento armazenado do PLPGSQL

Confira o comando:

Chamada a um procedimento armazenado do PLPGSQL.

Se fosse emitido o comando anterior, seria possível afirmar que as mensagens DEPARTAMENTO 10, DEPARTAMENTO 20 e DEPARTAMENTO 30 seriam exibidas, respectivamente, pelas seguintes quantidades de vezes:

A 1, 2 e 2.

B 2, 1 e 1.

C 2, 1 e 2.

D 1, 3 e 1.

E 1, 1 e 3.

Parabéns! A alternativa A está correta.

Como foi passado por parâmetro o número 5, serão lidos os empregados de ID 1 a 5 que são do departamento 10 o ID 1, do departamento 20 os de ID 3 e 5 e do departamento 30 os de ID 2 e 4, gerando os totais de 1, 2 e 2.

Resultado de processamento de um procedimento armazenado do PLPGSQL.

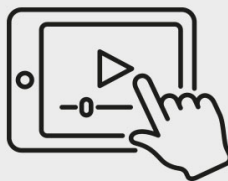
Procedimentos armazenados e PLPGSQL

Vamos agora conferir exemplos de uso dos comandos de programação vistos até agora. Para isso, iremos utilizar o banco de dados do museu, cujo modelo lógico é o seguinte:

Modelo de dados do estudo de caso.

Assista a este vídeo e veja como é possível implementar procedimentos armazenados.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade, confira o passo a passo:

1. Abra o pgAdmin4 e faça conexão no servidor.
2. Abra uma janela de consulta no banco do museu.
3. Realize as atividades descritas no arquivo de exemplos mod4nc3.

Você pode baixar os códigos que resolvem os pedidos no arquivo sqlmod4nc3.

Faça você mesmo!

Vamos agora executar uma atividade prática utilizando o banco de dados da escola. Observe o modelo lógico! Para realizar o exercício, faça conexão e realize as consultas solicitadas no exercício descrito no arquivo exercmod4nc3.

Modelo lógico do banco de dados escola.

Chave de resposta ▾

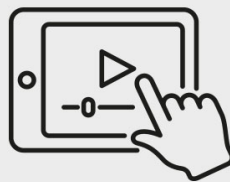
O arquivo da solução do exercício (solexermod4nc3.txt) também se encontra no material baixado.

Funções e gatilhos

Funções são tipicamente subprogramas que podem ser chamados em um comando de SQL e que retornam um valor em seu nome. O PostgreSQL possui um tipo especial de função para uso em gatilhos que são automaticamente executados em resposta a um evento de DML.

Confira neste vídeo de que forma devem ser implementados funções e gatilhos.

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Funções definidas pelo usuário

Funções são um tipo de subprograma que retorna um valor em seu nome. O PLPGSQL suporta dois tipos básicos de retorno: ESCALAR ou TABELA.

Funções com retorno escalar

A sintaxe para criar esse tipo de função é:

Estrutura para criação de FUNCTION no PLPGSQL.

Em caráter de exemplo, vamos criar uma função que receba o ID de um empregado e retorne o salário anual que corresponde ao salário do empregado multiplicado por 13 e acrescido de 1/3 de férias. Veja!

Criação de uma função.

Para ver a execução, comande:

Execução de uma função.

Funções com retorno tabela

A sintaxe para criar esse tipo de função é:

Estrutura para criação de FUNCTION no PLPGSQL.

Vejamos um exemplo em que a função recebe um cargo e retorna o ID e ULT_NOME do empregado.

Criação de uma função.

Para testar, execute o seguinte comando:

Execução de uma função.

Eliminando funções

O comando que elimina funções apresenta a seguinte sintaxe:

POSTGRESQL



CASCADE indica que devem ser eliminados os objetos dependentes da função, por exemplo, gatilhos.

O padrão é RESTRICT. Para eliminar a função F_EMP_RESUMO, o comando é:

Exclusão de uma função.

Gatilhos

Um gatilho do PostgreSQL é uma função invocada automaticamente sempre que ocorre um evento associado a uma tabela. Um evento pode ser qualquer um dos seguintes: INSERT, UPDATE ou DELETE.

Para criar um gatilho, você precisa primeiro definir a função de gatilho e, em seguida, vincular essa função a uma tabela.

Tipos de gatilho do PostgreSQL

O PostgreSQL fornece dois tipos principais de gatilhos:

1

Gatilhos em nível de linha

2

Gatilhos em nível de instrução

As diferenças entre os dois tipos têm relação com quantas vezes o gatilho é invocado e a que horas. Por exemplo, se você emitir uma UPDATE que modifica 20 linhas, o gatilho no nível da linha será invocado 20 vezes, enquanto o gatilho no nível da instrução será invocado 1 vez.

Invocação dos gatilhos

Você pode especificar se o gatilho é invocado antes (BEFORE) ou depois (AFTER) de um evento. Se o gatilho for invocado antes de um evento, ele pode ignorar a operação da linha atual ou até mesmo alterar a linha que está sendo atualizada ou inserida. Caso o gatilho seja invocado após o evento, todas as alterações estarão disponíveis para o gatilho.

Criando a função do gatilho

Uma função de gatilho é semelhante a uma função normal definida pelo usuário, sendo que ela não possui parâmetros de entrada e tem um valor de retorno do tipo trigger.

A sintaxe é:

POSTGRESQL



Uma função de gatilho recebe dados sobre seu ambiente de chamada por meio de uma estrutura especial, em que OLD.coluna e NEW.coluna representam os estados das colunas da linha na tabela antes ou depois do evento de disparo.

Depois de definir uma função de gatilho, você pode vinculá-la a um ou mais eventos de gatilho, como INSERT, UPDATE e DELETE.

Criando o gatilho

O comando que cria um gatilho possui a seguinte sintaxe:

POSTGRESQL



Em que você deve:

- Especificar o momento de disparo do gatilho, antes (before) ou depois (after) da execução do comando.
- Especificar o evento que chama o gatilho, podendo ser INSERT, DELETE ou UPDATE.
- Definir a tabela a qual o gatilho deve ser associado.
- Determinar o tipo gatilho linha (ROW) ou instrução (STATEMENT).
- Especificar o nome da função de gatilho a ser executada quando do disparo do gatilho.

Criando o gatilho

Vamos criar um gatilho na tabela EMPREGADO que faça auditoria na alteração do salário segundo a seguinte lógica: se o salário for alterado, deve ser inserida na tabela AUDIT_SALARIO o ID do empregado, o salário antigo, o novo salário e a data hora da alteração. Vamos começar criando a tabela AUDIT_SALARIO!

Criação da tabela AUDIT_SALARIO.

Vamos agora criar a função do gatilho.

Criação da função do gatilho.

Vamos agora criar o gatilho!

Criação do gatilho.

Para testar vamos dar um update de 1000 em todos os salários.

Teste do gatilho.

Você agora pode consultar a tabela e ver as linhas que foram inseridas.

Teste do gatilho.

Eliminando gatilho

O comando para eliminar um gatilho é o DROP TRIGGER, veja!

Eliminação do gatilho.

Atividade

Confira o conteúdo de algumas colunas da tabela empregado em um SGBD PostgreSQL!

Consultas da tabela EMPREGADO.

O PLPGSQL é uma linguagem procedural suportada pelo PostgreSQL que permite que sejam escritos gatilhos e que foram emitidos os seguintes comandos:

Comandos de criação de gatilhos.

Após a criação dos gatilhos, ao ser emitido o comando:

Comando SQL de exclusão.

A ordem correta de disparos dos gatilhos seria:

- A TG_EMP_GAC, TG_EMP_GAL, TG_EMP_GDC,
 TG_EMP_GDL
- B TG_EMP_GAC, TG_EMP_GDC, TG_EMP_GDL,
 TG_EMP_GAL
- C TG_EMP_GAL, TG_EMP_GDC, TG_EMP_GDL,
 TG_EMP_GAC
- D TG_EMP_GAC, TG_EMP_GAL, TG_EMP_GDL,
 TG_EMP_GDC
- E TG_EMP_GAL, TG_EMP_GAC, TG_EMP_GDL,
 TG_EMP_GDC

Parabéns! A alternativa D está correta.

O primeiro gatilho que dispara é o antes do comando - TG_EMP_GAC.

O segundo gatilho que dispara é o antes da linha - TG_EMP_GAL.

O terceiro primeiro gatilho que dispara é o depois da linha -
TG_EMP_GDL.

O último gatilho que dispara é o depois do comando - TG_EMP_GDC

Funções e gatilhos na prática

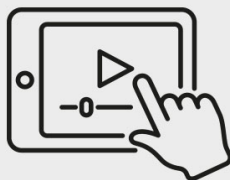
Vamos agora conferir exemplos de uso dos comandos de programação vistos até agora. Para isso, iremos utilizar o banco de dados do museu,

cujo modelo lógico é o seguinte:

Modelo lógico do banco de dados museu.

Neste vídeo, vamos demonstrar na prática como criar um banco de dados. Confira!

Para assistir a um vídeo sobre o assunto, acesse a versão online deste conteúdo.



Roteiro de prática

Para executar a atividade, siga o passo a passo:

1. Abra o pgAdmin4 e faça conexão no servidor.
2. Abra uma janela de consulta no banco do museu.
3. Realize as atividades descritas no arquivo de exemplos - mod4nc5.

Você pode baixar os códigos que resolvem os pedidos no arquivo sqlmod4nc5.

Faça você mesmo!

Vamos agora executar uma atividade prática utilizando o banco de dados da escola, cujo modelo lógico é apresentado na imagem. Baixe o script da criação das tabelas e inserção das linhas do banco de dados, crie o banco de dados da escola, faça conexão e realize as consultas solicitadas no exercício descrito no arquivo exercmod4nc5, que se encontram no material disponibilizado no campo **Preparação** no início deste conteúdo.

Modelo lógico do banco de dados Escola.

Chave de resposta ▾

O arquivo da solução do exercício (solexermod4nc5.txt) também se encontra no material baixado.

O que você aprendeu neste conteúdo?

Neste conteúdo, você viu:

1. Como instalar e acessar o PostgreSQL.
2. Como criar e povoar um banco de dados.
3. Como escrever consultas SQL para uma tabela.
4. Como escrever consultas SQL para mais de uma tabela.
5. Como escrever funções, procedimentos armazenados e gatilhos utilizando o PLPGSQL.

Explore +

Conheça melhor a história do PostgreSQL e da linguagem SQL no portal Wikipedia. Pesquise, respectivamente, por "PostgreSQL" e "SQL". Vale conferir!

Referências

BALIEIRO, R. **Banco de dados**. Rio de Janeiro: SESES, 2015.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

PostgreSQL. **Documentation**. Consultado na internet em: 24 maio 2023.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de banco de dados**. 5. ed. Rio de Janeiro: Campus, 2006.

Material para download

Clique no botão abaixo para fazer o download do conteúdo completo em formato PDF.

Download material

O que você achou do conteúdo?



 Relatar problema